

BeforeIT.jl: High-Performance Agent-Based Macroeconomics Made Easy

Adriano Meligrana^a, Mitja Devetak^{b,c,d}, Steven Hoekstra^{e,f}, Sebastian Poledna^g, Oliver Reiter^h, Aldo Glielmo^{i,*}

^aCENTAI, Turin, Italy

^bParis 1 Panthéon-Sorbonne University, Paris, France

^cComplexity Science Hub, Vienna, Austria

^dSupply Chain Intelligence Institute Austria, Vienna, Austria

^eUniversity of Amsterdam, Amsterdam, The Netherlands

^fTinbergen Institute, Amsterdam, The Netherlands

^gInternational Institute for Applied Systems Analysis (IIASA), Vienna, Austria

^hVienna Institute for International Economic Studies (wiiw), Vienna, Austria

ⁱApplied Research Team, Directorate General for Information Technology, Banca d'Italia, Rome, Italy^{**}

Abstract

BeforeIT is an open-source software for building and simulating state-of-the-art macroeconomic agent-based models (macro ABMs) based on the recently introduced macro ABM developed in [1] and here referred to as the *base model*. Written in Julia, it combines extraordinary computational efficiency with user-friendliness and extensibility. We present the main structure of the software, demonstrate its ease of use with illustrative examples, and benchmark its performance. Our benchmarks demonstrate that the base model built with BeforeIT is orders of magnitude faster than its original Matlab implementation, while achieving performance comparable to a highly optimized HPC implementation. BeforeIT is designed to facilitate reproducibility, extensibility, and experimentation. As the first open-source, industry-grade software to build macro ABMs, BeforeIT can significantly foster collaboration and innovation in the field of agent-based macroeconomic modelling. The package, along with its documentation, is freely available at <https://github.com/bancaditalia/BeforeIT.jl> under the AGPL-3.0.

Keywords: software package, agent-based model, simulation, macroeconomics

1. Introduction

Macroeconomic agent-based models (macro ABMs) are computational models in which the micro-level behaviour of heterogeneous interacting agents—such as households and firms—is simulated to produce a macro-level understanding of aggregate economic outcomes [2, 3, 4]. This modelling approach is in contrast with traditional macroeconomic frameworks based on dynamic stochastic general equilibrium (DSGE) models, which typically employ representative-agent assumptions and solve for equilibrium conditions under well-defined optimisation problems [5, 6, 7]. In comparison to DSGE models, macro ABMs use computational simulations of heterogeneous agents interacting, often requiring greater computational resources and more extensive calibration [8]. On the other hand, ABMs can naturally treat heterogeneity and nonlinear interactions among agents, as well as nonequilibrium dynamics.

Since their emergence in the early 2000s, macro ABMs have become an important tool for analysing



Figure 1: **Logo of the package.** BeforeIT stands for Behavioural economic forecasting by the IT research unit of the Bank of Italy. The agents in the ‘it’ monogram exemplify the agent-based nature of the simulation method. The name of the package also refers to the forecasting possibilities offered by the package, as in predicting a phenomenon *before it happens*.

economic dynamics, and particularly for reproducing stylized facts and capturing emergent phenomena such as asset bubbles, market crashes, economic divergence and convergence, and technological innovation [9, 3, 10, 11]. ABMs can also provide a strong foundation for policy analysis in areas such as systemic risk, housing market dynamics or inflation, and they have been increasingly employed by policy institutions [9, 12, 13, 4].

The macro ABM developed in [1] represents an im-

*Corresponding author, aldo.glielmo@bancaditalia.it

**The views and opinions expressed in this paper are those of the authors and do not necessarily reflect the official policy or position of Banca d'Italia.

portant step forward for the field as it demonstrated that ABMs can achieve short-term forecasting performances competitive with vector autoregressive (VAR) and DSGE models, alongside their capacity for reproducing emergent phenomena and stylized facts. The model’s flexibility and detailed representation of economic sectors has enabled it to address a variety of issues, ranging from the COVID-19 pandemic [14] and the 2021–2023 inflation surge [15] to financial crises [16], migration [17], natural disasters [18, 19], and labour productivity losses due to heat stress [20]. The model was extended to work in distributed memory parallel environments in [21], and was taken as the basis for the multi-country macro ABM developed in [22]. Interest in this model has grown among several institutions, including the Bank of Canada, which incorporated it into monetary policy analysis [23, 24, 25].

Over the years, this broad adoption has given rise to multiple implementations of such a model. The original implementation, available in open-source is written in Matlab [1, 26], and it has served as the foundation for other variants. For instance, the Bank of Canada created CANVAS—a production-oriented modification of the reference Matlab code—which is currently closed-source. While Matlab is a closed-source language, the original open-source Matlab code can also be run using the open-source Octave language, although this typically implies a significant slowdown in execution time. [21] introduced a Distributed Memory Parallel (DMP-HPC) implementation in C++ using MPI for supercomputers, which is likewise closed-source. Meanwhile, the University of Oxford [22] and Deloitte [27] have each developed their own closed-source extensions in Python and Java, respectively, underscoring the model’s adaptability across a range of environments.

Despite this array of implementations, until now, there was no user- and developer-friendly package that could be freely accessed, installed, and extended for diverse applications. To address this gap, we developed and released BeforeIT, an open-source, thoroughly documented software package that is both user- and developer-friendly, providing computational efficiency and a structure suitable for extensions. BeforeIT takes the model developed in [1] as its core reference, but it also offers a general platform that allows researchers to construct, modify, and extend macro ABMs tailored to their specific applications, thus fostering methodological experimentation and custom analysis. Since the model in [1] forms the foundation of BeforeIT, we refer to it as the base model in the rest of this work.

To ensure quality, reliability, and long-term usability, we made BeforeIT adhere to modern open-source pack-

age development standards and best practices. BeforeIT is publicly available on the GitHub page of the Bank of Italy¹, expanding the organization’s library of open-source ABM software packages, which includes other models [28] and a calibration engine [29, 30]. Its name and logo, illustrated in Fig. 1, highlight its focus on behavioural agent-based economic forecasting.

Developed in the Julia language, BeforeIT achieves computational efficiency comparable to C-compiled code while preserving readability and maintainability, characteristics typically associated with scripting languages such as Python or Matlab. Julia’s balance between ease of use and computational performance has contributed to Julia’s growing popularity in economics for numerical computations [31, 32], with notable adoptions by institutions such as the Federal Reserve [33, 34] and the Bank of Canada [35, 36].

By providing a platform to build, run and extend a model of proven impact, BeforeIT fills an important gap in the open-source software community with the aim to foster innovation, collaboration, and the wider adoption of macro ABMs.

The rest of this work is structured as follows. In Sec. 2 we briefly review the base model as this forms the foundation of BeforeIT, in Sec. 3 we describe the structure of the software and its main features, in Sec. 4 we provide practical examples of software usage, starting from an essential script and moving to more advanced scenarios, finally in Sec. 6 we conclude.

2. An essential model summary

The base model, the macro ABM developed in [1], simulates macroeconomic dynamics in a small open economy by modelling the interactions of millions of heterogeneous agents. The model is calibrated using data from national accounts, sectoral data, input-output tables, and business demographics, ensuring that the model reproduces exactly the state of the economy in that quarter in terms of aggregate GDP, GDP components, and industry sizes.

The model adheres to the European System of Accounts (ESA) framework [37] and organizes the economy into the following six classes of macroeconomic agents:

- **Households** make consumption, savings, and labour supply decisions.

¹<https://github.com/bancaditalia/BeforeIT.jl>

- **Non-financial corporations** produce goods using labour, capital, and intermediate inputs, with firm-level heterogeneity captured in terms of size, production processes, and market conditions.
- **Financial institutions** mediate credit markets.
- **The central bank** implements monetary policies.
- **The government** is responsible for taxation, public goods provision, and income redistribution
- **The rest of the world** accounts for external trade and international capital flows.

A distinguishing feature of this model in the literature of macro ABMs is its ability to provide aggregate macroeconomic forecasts—such as for GDP, inflation, and employment—and sector-specific forecasts. The rich micro-structure of the model, combined with adaptive learning, allows agents to form expectations based on past behaviours and observed outcomes. For example, households and firms rely on simple autoregressive (AR(1)) forecasting rules. Meanwhile, decentralised market interactions are characterised by search and matching mechanisms, which introduce trade frictions and market dynamics.

3. Software structure and main features

We should cite [38] for inspiration

Initialisation. To instantiate a model, it is sufficient to use its constructor function `Model`. This function takes two inputs: the dictionaries `parameters` and `initial_conditions`. The two dictionaries must be defined using specific keyword strings, such as `"tau_INC"` for the taxation rate on income or `"mu"` for the risk premium over the policy rate charged by commercial banks to issue loans. We use the same keywords used in the original parametrisation, available from the supplementary material of [1]. This ensures full compatibility with the Matlab parametrisations provided by the original paper, allowing users to easily load the Matlab file and use the resulting variables for initialization. For quick experimentation, we include in the package the original parametrisation for Austria in the first quarter of 2010. We also provide an experimental parametrisation for Italy. However, we believe that the generation of parameters and initial conditions should ultimately be handled in a dedicated package specifically tailored to this purpose.

Structure. The different classes of macroeconomic agents described in the previous section are implemented in `BeforeIT` as separate structure of arrays (SoA) objects, which in Julia are defined via “structs”.

Specifically the package defines the following objects: `w_act` (for households consisting of active workers), `w_inact` (for households consisting of inactive workers), `firms` (for non financial corporations), `bank` (for the financial sector), `cb` (for the central bank), `gov` (for the government), `rotw` (for the rest of the world). Each object contains specific and thoroughly documented attributes corresponding to the key features of the respective agent type. For instance, the `w_act` struct includes an attribute `Y_h`, which stores the income of all active workers at any given simulation time-step. The attribute names across all agent types are chosen to align closely with the equations and notation used in the original publication [1] to facilitate a clear connection between the code and the theoretical model.

Finally, the entire model is encapsulated within a struct named `model`, which includes the seven agent groups as its attributes, along with three additional objects: the `prop` struct, which stores the properties of the simulation run, the `agg` struct, which holds the aggregate variables representing the overall state of the national economy being simulated, and the `data` struct, which holds the data collected during the simulation run. The left side of Figure 2 visually illustrates this hierarchical structure. The `model` struct is represented as a large rectangle containing nine smaller rectangles, corresponding to the seven agent classes and the two supplementary objects, `prop` and `agg`.

Inspection. After a model is created as described above, its attributes can be inspected at any time using the common ‘dot’ notation. For example, the latest value of the policy rate can be printed using `println(model.cb.rate)`, as illustrated in Figure 2. This feature can be especially convenient in interactive scripts or Jupyter notebooks to quickly verify the effects of specific code changes or interventions in the model.

Simulation. Once a model is created using the appropriate function, a simulation can be executed with a single line of code by calling the function `run!`. This function takes the model object as input and returns the updated model after the simulation run. Note that, in Julia, the exclamation mark at the end of a function denotes a function that changes its arguments in-place. In this case, `run!` changes the model in-place, and hence the `model` object after the call stores the final state of the model. In contrast to `step!`, the `run!` function collects data after each step. The `data` object has well-documented, intuitively named attributes, such as `nominal_gdp` or `real_household_consumption`. By default, 25 variables are tracked, but the set can be expanded by

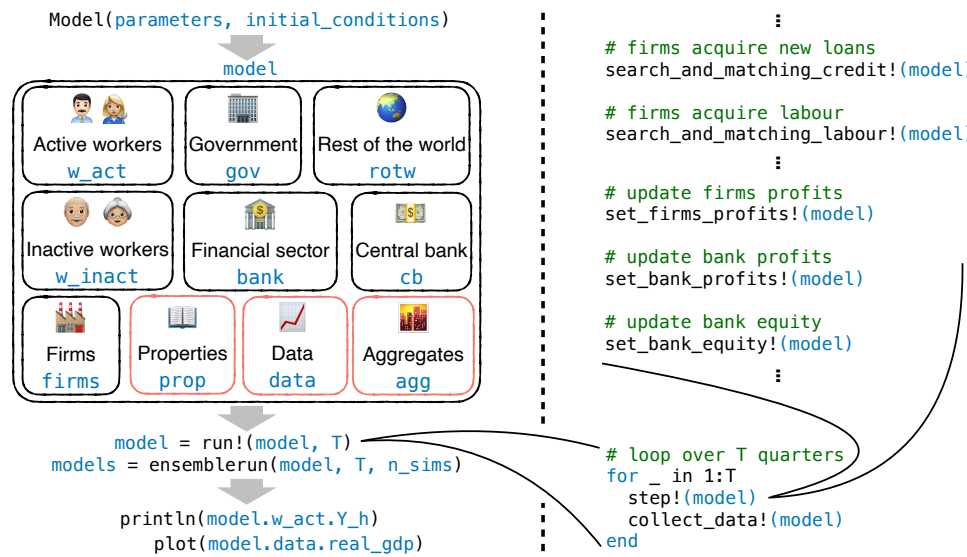


Figure 2: **Illustration of the workflow and main objects of the software.** The left-hand side illustrates the typical workflow of a simulation run. The function `init_model` takes as input the two dictionaries `parameters` and `initial_conditions` and the number of simulation steps `T`, returning an object named `model`. This `model` object is composed of distinct objects representing the different agents of the economy, such as `gov` and `firms` and two other objects that do *not* represent agents but store properties and aggregate variables, i.e., `prop` and `agg`. The `model` object serves as input for `run!` or `enssemblerun`, where the second function performs multiple independent Monte Carlo runs. Simulation results are stored in a `data` object for a single run or in a `data_vector` object for multiple runs, both of which allow easy inspection and visualization of attributes for any agent type. Similarly, we can inspect any attribute of any of the agent classes. The right-hand side illustrates the fact that a simulation run is essentially a for loop over the function `step!` and that a run over one step consists of a series of well-defined function calls taking the different agent types as input and updating specific attributes of the same agent type, making up modular and transparent simulation process.

customising the data tracker. All variables in the `data` object can be inspected using dot notation, for example, by calling `plot(data.real_gdp)`. It is also worth noting that the function for running a single simulation includes a keyword argument, `parallel`, which defaults to `false`. This option allows the consumption goods market for different industrial sectors to run on separate threads, utilising as many threads as are available to the Julia session. It is also straightforward to run a large number of Monte Carlo repetitions of the same simulation using the function `enssemblerun`. This function takes the `model` and the number of runs as input and returns a vector of `data` objects that can also be inspected using dot notation. A keyword parameter, `parallel`, is available also for this function and defaults to `true`. The multi-threading here is more efficient than the intra-model multi-threading across industrial sectors, as the threads operate on independent runs.

Modularity. The right-hand side of Figure 2 illustrates the inner workings of the model runs. Specifically, the figure shows that the `run!` function is essentially a simple for loop over a desired number of quarters, `T`, of the function `step!`, combined with a function to up-

date the data tracker with the latest model data. The figure also presents an excerpt from `step!`, highlighting how the main simulation loop consists of a sequence of very readable functions. Each function takes as input the agent type performing the corresponding action, as well as the entire `model`, allowing the agent type to interact with it, and returns the updated value of specific attributes. For example, by this convention, we have that `bank_profits` takes `bank` and `model` as input and returns the updated bank profits, which are then assigned to the attribute `bank.Pi_k`. This clear modular structure allows for precise changes and extensions to the `model` by simply redefining specific functions.

Speed. In addition to being easy to use and extend, BeforeIT is highly computationally efficient. In Figure 3 we show the efficiency of the software by benchmarking the mean time required to run the model of Austria in scale 1:1000, hence simulating around 8000 agents, and the model in scale 1:1, with around 8 million agents, for one step against the Matlab implementation, the C implementation generated with the Matlab-Coder toolbox and the HPC model developed in [21]. Our tests show that BeforeIT is approximately 20 times faster than the Matlab implementation and 4 times faster than

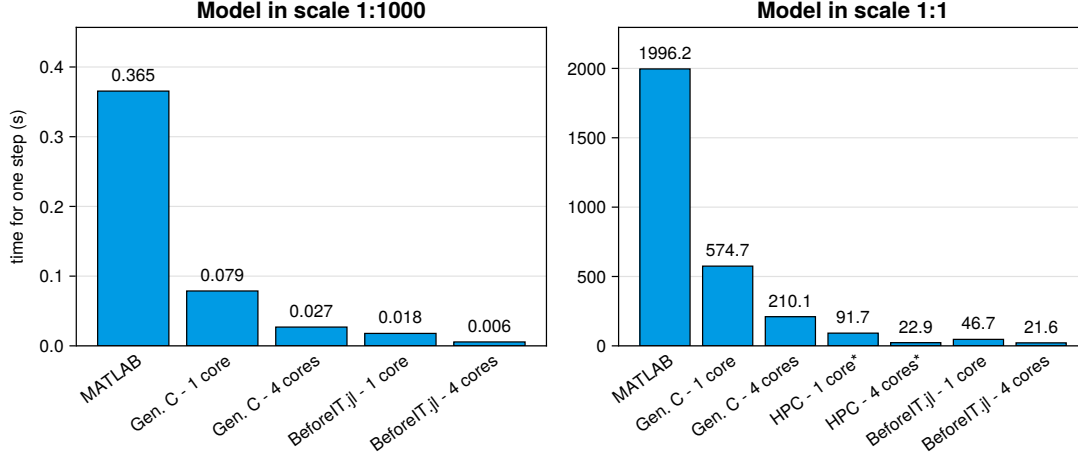


Figure 3: **A benchmark of the computational efficiency of different implementations.** The figure shows the mean time to run one step of the model calibrated on the Austrian economy with around 8 thousand agents and 8 million agents using the original Matlab code, the C code generated by the Matlab Coder toolkit, the HPC implementation and BeforeIT. The benchmarks were executed on Linux x86_64 with an AMD Ryzen 5 5600H CPU and 16 GB of RAM, except for the HPC results, which were only estimated by using the timings in [21].

the Matlab-generated C code for the small version in the single-threaded case and, respectively, 43 times and 12 times faster when 8 million agents are simulated. Our simulation appears to have better run-time performance compared to the HPC implementation of the model by looking at the running times reported in the original article [21]. However, given the unavailability of the source code and the different execution environments, we cannot measure the exact speed-up. The efficiency of BeforeIT is largely due to Julia’s just-in-time compilation, which enables a performance comparable to C while making the code significantly easier to develop, maintain and understand [39, 40, 41]. Additional performance gains are achieved by employing more optimized algorithms than those used in the original Matlab version. The greatest part of the performance improvement was made possible by using dynamic weighted sampling techniques [42, 43]. In particular, we used the state-of-the-art implementation provided in [44], which allowed to simulate the market exchange process for goods much more efficiently. Notably, even without the use of such advanced algorithms, our software proved to be approximately on par with the C implementation. The figure also illustrates that applying multi-threading across different industrial sectors can further accelerate the simulation, albeit the speed-up is sub-linear relative to the number of cores employed; even if we believe this aspect could be improved in the future, the scalability of the HPC implementation is still better than what BeforeIT.jl currently achieves. However, when parallelizing over different independent Monte Carlo runs,

BeforeIT.jl offers the best performance, without sacrificing ease of use.

Testing. BeforeIT includes a large and expanding suite of *unit tests* to ensure its quality and reliability. The current set of unit tests can be divided into three classes. At the lowest level, we test specific functions representing individual agent actions against the expected behaviour. As an example, we test that the function `taylor_rule` with specific values of input arguments returns the same number that one can compute with pen and paper. At the aggregate level, we test that the code respects several accounting identities. For instance, we check that the national income identity holds after every step, i.e., that the GDP equals the sum of all expenditures. Finally, at the model level, we test that a deterministic version of the model exactly matches the detailed behaviour of a deterministic version of the original Matlab code. Additionally, stochastic versions of the two models are rigorously tested for coherence through statistical analysis.

These layers of tests are run automatically through GitHub’s continuous integration workflows every time the code is modified to collectively ensure both the accuracy and robustness of the software.

Documentation. BeforeIT is thoroughly documented. The documentation for the main functions governing the inner workings of the package and of the agents in the model is embedded in the source code and also forms an API reference available at <https://bancaditalia.github.io/BeforeIT.jl>. The webpage provided also includes

```

import BeforeIT as Bit
import Plots, StatsPlots

# load parameters and initial conditions
p = Bit.AUSTRIA2010Q1.parameters
ic = Bit.AUSTRIA2010Q1.initial_conditions

# initialise a model object
model = Bit.Model(p, ic)

# define a simulation length
T = 20

# run 8 simulations in parallel
data_vector = Bit.ensamplerun(model, T, 8)

# plot the results
plots = Bit.plot_data_vector(data_vector)
Plots.plot(plots...)

```

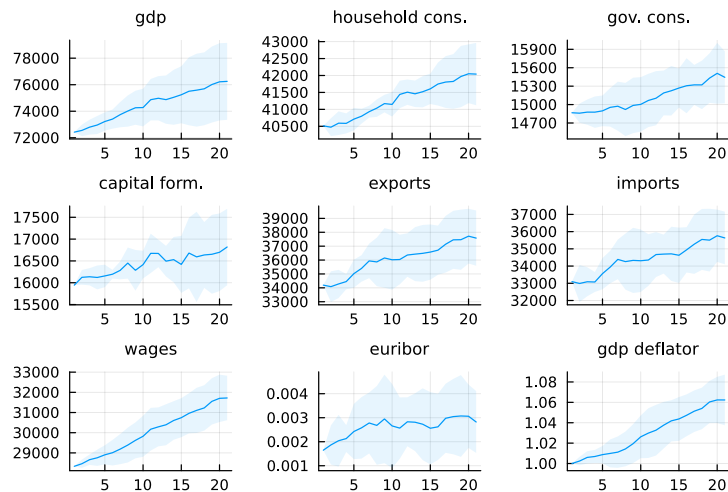


Figure 4: **A simple script and its resulting graphs.** The left panel shows how to run the model with just a few lines of code. We import the package, load the original parametrisation for Austria in the first quarter of 2010, and specify a given number of quarters to simulate. Then we call the `Model` function and then the `ensamplerun` function, specifying 8 Monte Carlo runs. Some of the results contained in the resulting `data_vector` are provided on the right panel.

several tutorials to quickly learn how to use the package with a hands-on approach. The source code reported in the online tutorials is available in the “examples” folder of the GitHub repository along with other example usages.

4. Usage illustration

Here, we present some illustrative examples of how to use the package.

Essential usage. Figure 4 provides a step-by-step example of typical usage of the package. The left-hand side outlines the exact commands required to load the package, initialise a model, run different Monte Carlo runs of the model, and then inspect the results. The right-hand side presents a few plots of the simulation runs, with the shaded area indicating the standard error on the mean of the Monte Carlo repetitions.

Shocked simulations. We also provide a high-level interface for simulating arbitrary economic shocks to the model. This functionality is achieved through a “shock” object, implemented as a callable struct.

At the beginning of every step, and before agents’ actions, the shock object is invoked. The shock object receives the entire model as input, enabling it to modify any model attribute in-place in a highly flexible manner. The left panel of Figure 5 illustrates this process with an example of a consumption shock. In the figure, we first create a struct with two attributes. Then we make the struct “callable” by defining a function on top of

the struct. The function increases the propensity to consume `psi` by the factor `multiplier` until the simulation time `model.agg.t` reaches `final_time`, when it is set back to the original level. Finally, a specific shock (`my_shock`) is instantiated with a 2% increase in the propensity to consume for 4 quarters. This shock is then passed as a keyword argument to the function `ensamplerun`. By default, the keyword argument is set to a dummy “NoShock” struct that performs no actions. This abstraction allows for arbitrary complex shocks to be implemented straightforwardly since both the state of the model and any of its parameters are accessible as attributes of the `model` object and can be easily manipulated.

Calibration to EU nations. In a separate project called `CalibrateBeforeIT.jl`² we provide functions and exemplary scripts which allow the user to generate the necessary parameters and initial conditions for other EU nations.

The code in the `CalibrateBeforeIT.jl` package is based on the Matlab and PostgreSQL scripts used in [1] (which were already extended to work for all EU27 countries at that point) and were kindly shared with us. To ease the accessibility of this work, we translated the scripts to Julia, updated Eurostat sources and made the

²The `CalibrateBeforeIT.jl` is currently available at <https://github.com/ViennaInstitute/CalibrateBeforeIT.jl>. In the long term our goal is to integrate its functionality within the `BeforeIT` framework.

```

# define a custom shock
struct ConsumptionShock <: Bit.AbstractShock
    multiplier::Float64
    final_time::Int
end

# define the shock behaviour
function (s::ConsumptionShock)(model)
    if model.agg.t == 1
        model.prop.psi *= s.multiplier
    elseif model.agg.t == s.final_time
        model.prop.psi /= s.multiplier
    end
end

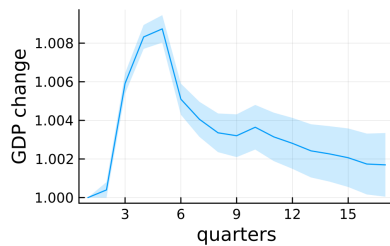
# load parameters and initial conditions
p = Bit.AUSTRIA2010Q1.parameters
ic = Bit.AUSTRIA2010Q1.initial_conditions

# set shock parameters
my_shock = ConsumptionShock(1.02, 4)

# initialise model
model = Bit.Model(p, ic)

# run the shocked model
T = 20
data_vec = Bit.ensamplerun(
    model, T, 64; shock = my_shock
)

```



```

# define a new model object
Bit.@object struct NewModel{Bit.Model} end

# define a central bank object with extra attributes
Bit.@object mutable struct NewCB{Bit.CentralBank}
    avg_rate::Float64
end

# change the default central bank behaviour
function Bit.set_central_bank_rate!(model::NewModel)
    model.cb.r_bar = model.cb.avg_rate + 0.1*randn()
end

# initialise all basic model components
prop = Bit.Properties(p, ic)
firms = Bit.Firms(p, ic)
w_act, w_inact = Bit.Workers(p, ic)
bank = Bit.Bank(p, ic)
gov = Bit.Government(p, ic)
rotw = Bit.RestOfTheWorld(p, ic)
agg = Bit.Aggregates(p, ic)
data = Bit.Data()

# initialise the custom central bank
cb = Bit.CentralBank(p, ic)
newcb = NewCB(Bit.fields(cb)..., 0.02)

# initialise model
model = Bit.Model(w_act, w_inact, firms, bank, newcb,
    gov, rotw, agg, prop, data_fast)

# run the extended model
T = 20
data_vec = Bit.ensamplerun(model, T, 8)

```

Figure 5: **Scripts for shocks and for extensions.** The left-hand side script illustrates the usage of the interface to run simulations with shocks. In essence, one simply needs to define a callable struct and pass it to the function `ensamplerun` as the `shock` keyword argument. The effect of the simulated consumption shock on the GDP (ratio of shocked over baseline) is presented in the graph at the bottom. The right-hand side script illustrates the usage of macros and methods' specialization to extend the model without needing to copy-paste code. In essence, one can create a new mutable struct for a custom central bank as a subtype of `AbstractCentralBank` and inherit all standard attributes using the `@centralBank` macro. Then, one can modify specific behaviours by specializing some functions for `cb::NewCentralBank`, the other functions will remain unchanged.

process more robust against missing data. The only prerequisite for this repository is Julia, there are no outside technical dependencies.

We showcase the usage of the package in three scripts (available in the repository). First, usage of the helper functions to a) download data from a data snapshot at Zenodo³ to be able to replicate our results or b) download the latest available data from Eurostat. The second script shows the application of some necessary data transformations (e.g. appending two data sources to have longer timeseries available). Finally, a third script

shows the helper functions that are used to query the data sources for the timeseries necessary to construct the initial conditions and parameters. At our Zenodo repository, we also provide the resulting country calibrations (which can be used to calculate initial conditions for the BeforeIT model).

At the moment of writing, the `CalibrateBeforeIT.jl` package works for all EU nations except **OR: Malta and Luxembourg TODO check for all quarters from 2015 to 2023**. Applying the calibration process for years before 2015 is possible, but more prone to fail due to missing data.

We are actively developing the `CalibrateBeforeIt.jl`

³See <https://doi.org/10.5281/zenodo.18698434>

package to extend its country and time coverage and remedy any existing inconsistencies.

Automatic differentiation.

- What is AD and where it comes from (deep learning).
- Research frontier: AD for ABMs

Julia is uniquely well-suited for automatic differentiation (AD) due to its language design, which has fostered a diverse ecosystem of specialized AD tools. These tools generally fall into distinct methodological categories. At the lowest level, Enzyme.jl [45, 46] provides first-class bindings to the Enzyme compiler, enabling highly efficient differentiation directly at the LLVM Intermediate Representation (IR) level. At the language level, source-to-source AD compilers transform Julia IR to compute gradients: Zygote.jl [47] serves as a foundational tool in this space, though it exhibits certain expressivity limitations that newer systems, such as Mooncake.jl, aim to resolve by supporting differentiation across arbitrary Julia programs. Alternatively, packages like ForwardDiff.jl and ReverseDiff.jl utilize operator overloading. While restricted to a narrower subset of differentiable programs compared to compiler-based approaches, their underlying implementations are less complex and highly stable. Finally, the ecosystem extends to specialized domains, such as the differentiation of programs with discrete random variables, currently supported experimentally by StochasticAD.jl.

Currently, BeforeIT can be automatically differentiated using the Mooncake compiler. Achieving this compatibility required refactoring specific segments of the codebase to circumvent non-differentiable code blocks. For instance, because Mooncake cannot differentiate through foreign function calls (such as LAPACK routines), the singular value decomposition (SVD) used to estimate expected GDP and inflation had to be re-implemented in pure Julia. While currently unsupported, future integration with Enzyme would be highly beneficial for cross-validating gradient computations across distinct AD backends.

To verify the effectiveness of using automatic differentiation, we run a simple optimization procedure on the GDP. We tried to optimize the following objective:

$$\frac{GDP(\theta)_{t=2} - GDP(\theta)_{t=1}}{GDP(\theta)_{t=1}} - 100 \cdot \left(\frac{GDP(\theta)_{t=1} - GDP(\theta_0)_{t=1}}{GDP(\theta_0)_{t=1}} \right)^2$$

where $GDP(\theta_0)_{t=1}$ represents the initial GDP of the original Austrian parametrization, while $GDP(\theta)_{t=1}$ and $GDP(\theta)_{t=2}$ are the values of the GDP represent the initial and final GDP of the new parametrization respectively. To make the gradient optimization more stable,

only the 17 tax parameters of the model have been allowed to vary in the optimization.

We compared the results of optimizing the model through numerical gradients computed using central finite differences and the automatic gradient extracted with Mooncake.jl. Noticeably, using standard 64-bit floating point numbers for the numerical gradient was too numerically unstable making the optimization fail. Instead, running the optimization using 128-bit floating point numbers, made possible by the support of BeforeIT to run the model with any floating point type, allowed the optimization proceed smoothly. Without any adjustment, instead, the automatic gradient ran the optimization almost as accurately as the 128-bit numerical gradient:

figures/AD_optimization.png

Figure 6: **GDP Optimization Trajectories by Gradient Method.**

Performance-wise, computing gradients through automatic differentiation makes an enormous difference.

Extensions. We include in BeforeIT specific macros that allow for easily extending the original agent classes without copying and pasting the entire codebase. In the right panel of Figure 5 we provide an example of how to leverage these macros. Suppose we want to extend the model by introducing a new type of central bank with custom behaviour. We do this by creating a custom struct called `NewCB`. Crucially, we define this to be a subtype of `AbstractCentralBank`. This allows the new struct to seamlessly inherit all existing functions defined in BeforeIT for central banks.

In our new struct we want to inherit all existing

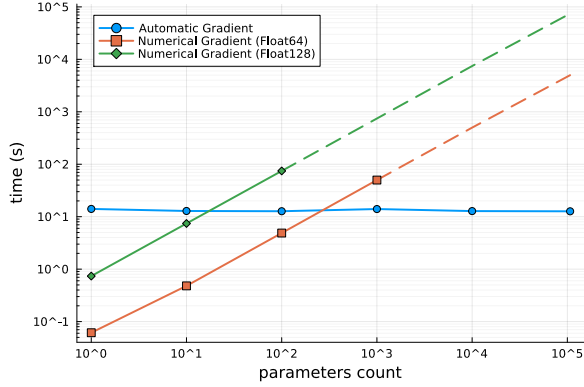


Figure 7: **TODO.**

attributes of the standard central banks in the package and add additional attributes. We do this using the macro `@object`, which essentially copy-pastes all the attributes of the inherited object, which here is `CentralBank`, within the newly created one. Additional attributes can then be defined as needed. Finally, we customise the behaviour of the central bank by defining a new `central_bank_rate` function. In this function, we specify that `cb` is of type `NewCB`. In this way, through Julia’s powerful multiple-dispatch mechanism, the compiled code will use this function instead of the default one, and will instead use the default functions for any other behaviours of the central bank as provided in the package implementation.

Now what is left is to change the initialisation of the model by allowing our new object to appear. To do so, we explicitly initialise each agent type and overwrite the default central bank with our extended object.

5. Systematic model improvements

The extension mechanism introduced in Section 4 enables more than ad-hoc experimentation: it provides a principled framework for *systematic* model improvement. By pairing model variants with an automated multicountry forecasting pipeline, researchers can rigorously evaluate alternative modelling assumptions across all calibrated economies using standardised error metrics. This section describes the two main ingredients: (i) how full model variants are defined within `BeforeIT`, and (ii) how they are evaluated at scale, illustrating the workflow with a concrete example.

Defining model variants. A model variant in `BeforeIT` is a self-contained Julia file that defines new agent types and overrides specific behavioural functions via

multiple dispatch, leaving the rest of the simulation unchanged. The `@object` macro copies all fields from a parent type and appends any additional ones, while the Julia type system ensures that only the overridden functions are replaced. Two representative variants are shipped with the package.

The first, `ModelCANVAS`, implements the demand-pull pricing mechanism of [23] by overriding three functions: `firms_expectations_and_decisions`, which introduces firm-level quantity and price adjustments based on excess demand; `central_bank_rate`, which replaces the fixed-parameter Taylor rule with an adaptive version that re-estimates its coefficients each period; and `growth_inflation_EA`, which records the euro-area time series needed for the adaptive estimation. These overrides require two extended agent types: `CentralBankCANVAS`, which stores the historical interest rate series, and `RestOfTheWorldCANVAS`, which stores euro-area GDP and inflation histories. All other agents (firms, households, banks, government) remain unchanged.

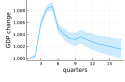
The second variant, `ModelGR`, replaces the log-level AR(1) processes used for expectations and exogenous drivers with growth-rate AR(1) specifications,

$$g_t = \alpha g_{t-1} + \beta + \varepsilon_t, \quad X_t = X_{t-1} \cdot (1 + g_t), \quad (1)$$

where g_t denotes the growth rate of the variable X_t . This affects GDP expectations, inflation expectations, government consumption, and trade volumes, and requires two new agent types, `GovernmentGR` and `RestOfTheWorldGR`, that store the estimated AR(1) coefficients and lagged growth rates. In both cases, the core simulation loop (`step!`) remains completely untouched. Julia’s compiler automatically dispatches to the overridden methods based on the model’s concrete type, ensuring that every variant is evaluated under identical market-clearing, accounting, and aggregation rules and thereby isolating the effect of the modelling change.

Multicountry forecasting pipeline. To evaluate model variants at scale, `BeforeIT` includes an automated pipeline that runs out-of-sample forecasting exercises across all calibrated countries. A variant is identified by a name string and a model constructor; output is automatically organised under `data/{country}/analysis/{variant}/`, so adding a new variant requires no changes to the pipeline code itself.

The pipeline has two phases, both driven by the same script:



```

model_variant = "growth_rate"
model_factory = ModelGR
# Simulation phase
for country in Bit.discover_countries_with_calibration()
    calibration = Bit.load_calibration_data(country)
    Bit.run_variant_pipeline(
        "data/\$country", calibration.data, model_variant;
        model_factory, T = t, n_sims)
end
# Analysis phase
for country in Bit.discover_countries(model_variant)
    calibration = Bit.load_calibration_data(country)
    Bit.error_table_ar(country, calibration.ea,
        calibration.data, quarters, horizons; model_variant)
    Bit.error_table_abm(country, calibration.ea,
        calibration.data, quarters, horizons; model_variant,
        prediction_folder = "abm_predictions/\$model_variant")
end

```

Figure 8: **AG: for some reason I am unable to put the two figures side by side (the first one is a place-holder) I will try and fix it**

In the *simulation phase*, `run_variant_pipeline` iterates over all starting quarters from 2010Q1 to 2019Q4 for each country, initialising the model from its calibration data, running N Monte Carlo simulations of length T using the provided `model_factory`, and saving forecast trajectories to disk. When `model_factory` is omitted, the standard Model constructor is used, corresponding to the base model.

In the *analysis phase*, forecast accuracy is assessed using the root mean squared error (RMSE) and bias at horizons of 1, 2, 4, 8, and 12 quarters for five key macroeconomic variables: real GDP, GDP deflator growth, real household consumption, real investment, and the Euribor rate. Statistical significance of the bias is tested using a t -test with Newey–West standard errors to account for serial correlation in forecast errors. Model comparisons are performed via the Diebold–Mariano test, which assesses whether the difference in squared forecast errors between two models is statistically significant. Results are stored per country in `data/{country}/analysis/{variant}/` and then aggregated across countries to produce summary tables and heatmaps.

Example results.

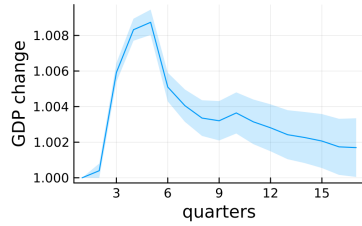
Figure 10 illustrates the kind of output the pipeline produces, showing the RMSE difference between `ModelGR` and a univariate AR benchmark across all calibrated EU countries. Each cell reports $(RMSE_{AR} - RMSE_{ABM}) \times 100$, so that positive values (blue) indicate that the ABM outperforms the AR and negative values (red) indicate the reverse; significance stars denote re-

jection of equal predictive accuracy under the Diebold–Mariano test.

The results are heterogeneous across countries and variables. The ABM achieves notably better forecast accuracy for real investment in several countries (e.g., the Netherlands and Bulgaria), while the AR benchmark tends to be more competitive for real GDP and inflation in many economies. Outliers such as Estonia’s real consumption reflect episodes of large structural change that are difficult to capture with any fixed specification. This heterogeneity illustrates the value of running the pipeline at scale: patterns that would be invisible in a single-country study become apparent when results are aggregated.

6. Conclusions

This paper introduces `BeforeIT`, a software package to build state-of-the-art macroeconomic agent-based models (macro ABMs) that are based on the recently introduced base model [1], in a fast, user-friendly, and extensible framework. By leveraging the computational efficiency and flexibility of the Julia programming language, `BeforeIT` delivers high-performance code that is greatly superior to existing implementations of the base model, such as Matlab or Matlab-generated C code, while maintaining simplicity for users and developers. Its modular structure, thorough documentation, and extensive testing ensure that the package can be easily picked up and used for applications ranging from policy analysis and economic forecasting to any model extension. As the first open-source package to build mod-



```

model_variant = "growth_rate"
model_factory = ModelGR
# Simulation phase
for country in Bit.discover_countries_with_calibration
()
    calibration = Bit.load_calibration_data(country)
    Bit.run_variant_pipeline(
        "data/$country", calibration.data,
        model_variant;
        model_factory, T = t, n_sims
    )
end

# Analysis phase
for country in Bit.discover_countries(model_variant)
    calibration = Bit.load_calibration_data(country)
    Bit.error_table_ar(
        country, calibration.ea, calibration.data,
        quarters, horizons; model_variant
    )
    Bit.error_table_abm(
        country, calibration.ea, calibration.data,
        quarters, horizons; model_variant,
        prediction_folder = "abm_predictions/$
        model_variant"
    )
end

```

Figure 9: CAPTION

els like [1], BeforeIT fills an important gap in making the latest macro ABMs more accessible. We believe the package can greatly facilitate experimentation, e.g., involving calibration using large volumes of data, or extensive sensitivity analyses and forecasting. The existing open-source Julia ecosystem for data science and numerical simulation can be of great help for these projects. The Python open-source ecosystem could be equally leveraged as it is possible to bridge the two languages seamlessly using libraries such as PythonCall and JuliaCall [48]. Most generally, BeforeIT can contribute to the foundation of the next-generation tools for macro ABM research.

We foresee several lines of development that could further enhance BeforeIT. One important enhancement is the modularisation and standardisation of the calibration scripts. Differently from the rest of the package, these are not modularly written and currently come in the form of an unpolished research prototype. However, they could be easily improved and, potentially, even be adapted to make them work for any European nation. The resulting code could be integrated within BeforeIT or be released as a dedicated companion package. A

second line of development would involve the inclusion in the package of other models that were built on top of the base model. For example, it should be straightforward to include the so-called ‘CANVAS’ model, built by the Bank of Canada, by appropriately adjusting the behaviour of specific agents as described in the corresponding publication. Finally, it would be interesting to endow the package with automatic differentiation. Automatic differentiation is available in Julia through several libraries and the inclusion in BeforeIT is certainly possible by appropriately extending functions in the code that are non-smooth. This, in turn, would enable the possibility to perform sensitivity analysis just by looking at the gradient of the model output, and it could help in calibrating the model to aggregate output data if deemed necessary or useful.

For any of the mentioned extensions or any other improvement to the package, we greatly encourage open-source contributions to the project in the form of issues, pull requests or feedback.

sk	24.2***	-17.3***	11.8	2.1	-3.5
si	-1.4	6.0***	17.8***	2.3	-5.5
se	-5.4	-20.8**	-78.6***	-22.2**	-8.1
ro	-42.5***	-35.5*	-18.1	35.5***	-1.9
pt	11.6***	-3.9***	13.1***	12.2***	-5.4
pl	-0.1	-67.1***	7.4	21.6**	0.7
nl	-32.8***	5.1	-27.1***	81.1***	-2.7
lv	27.5*	-28.8***	24.3	26.4***	-4.6
lt	-93.7***	-27.7*	-124.6***	-51.4***	-7.9
it	-37.5***	-6.9	-8.6	-0.1	-3.0
ie	-8.8***	-14.3**	31.9**	0.1	-1.7
hu	-11.2*	-46.8***	-6.8	-1.8	-7.6
hr	-31.0*	-8.7	-16.4	-24.4***	-87.5***
fr	-5.5	-16.2*	36.2***	6.3	-2.3
fi	6.7	-19.7***	40.5***	2.8	-3.6
es	-24.6***	0.5	3.9***	-4.4	-2.1
el	-16.1***	1.3	-5.8***	-7.1*	-2.9
ee	-75.9***	-32.2***	-418.1***	-1.8	-2.0
dk	-5.1	-7.6	-81.7***	-8.9***	-2.2
de	-52.8***	-13.6***	-52.9**	26.5***	-0.1
cz	-13.0	-24.2***	-18.5***	6.1	-3.9
cy	3.3	-7.2***	-0.9	2.3	-2.1
bg	-34.1***	-110.3***	-9.0	62.1***	-5.7
be	-19.7**	-2.8***	-203.3***	-39.3***	-12.1**
at	-34.7***	-9.7	-0.4	12.7*	3.8

Figure 10: **RMSE improvement of ModelGR over a univariate AR benchmark.** Each cell shows $(RMSE_{AR} - RMSE_{ABM}) \times 100$ at a four-quarter horizon, averaged over all starting quarters from 2010Q1 to 2019Q4. Blue cells indicate that the ABM produces lower forecast errors than the AR; red cells indicate the reverse. Significance stars denote rejection of the null of equal predictive accuracy under the Diebold–Mariano test (* $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$).

Acknowledgments

AG would like to thank the following colleagues from Banca d'Italia: Marco Benedetti, Claudia Biancotti and Marco Favorito for their continuous help, support and feedback on the project, Andrea Gentili and Sara Corbo respectively for their help on the design of the package's name and logo. The authors thank Claudia Biancotti (Banca d'Italia), Pietro Terna (University of Turin) and Marco Pangallo (CENTAI institute) for early feedback on the manuscript. SP acknowledges funding from the Vienna Science and Technology Fund (WWTF) (grant number ESS22-040). The views and opinions expressed in this paper are those of the authors and do not nec-

essarily reflect the official policy or position of Banca d'Italia.

References

- [1] S. Poledna, M. G. Miess, C. Hommes, K. Rabitsch, Economic forecasting with an agent-based model, *European Economic Review* 151 (2023) 104306.
- [2] L. Hamill, N. Gilbert, *Agent-based modelling in economics*, John Wiley & Sons, 2015.
- [3] H. Dawid, D. Delli Gatti, *Agent-based macroeconomics*, *Handbook of computational economics* 4 (2018) 63–156.
- [4] R. L. Axtell, J. D. Farmer, *Agent-based modeling in economics and finance: Past, present, and future*, *Journal of Economic Literature* (forthcoming) (2024).
- [5] L. J. Christiano, M. Trabandt, K. Walentin, DSGE models for monetary policy analysis, in: *Handbook of monetary economics*, Vol. 3, Elsevier, 2010, pp. 285–367.
- [6] M. Del Negro, F. Schorfheide, DSGE model-based forecasting, in: *Handbook of economic forecasting*, Vol. 2, Elsevier, 2013, pp. 57–140.
- [7] L. J. Christiano, M. S. Eichenbaum, M. Trabandt, On DSGE models, *Journal of Economic Perspectives* 32 (3) (2018) 113–140.
- [8] M. Pangallo, R. M. del Rio-Chanona, Data-driven economic agent-based models, *arXiv preprint arXiv:2412.16591* (2024).
- [9] A. Turrell, *Agent-based models: understanding the economy from the bottom up*, *Quarterly Bulletin Q4*, Bank of England (2016).
- [10] G. Dosi, A. Roventini, More is different... and complex! the case for agent-based macroeconomics, *Journal of Evolutionary Economics* 29 (2019) 1–37.
- [11] H. Dawid, D. Delli Gatti, L. E. Fierro, S. Poledna, Implications of behavioral rules in agent-based macroeconomics (2024).
- [12] R. Baptista, J. D. Farmer, M. Hinterschweiger, K. Low, D. Tang, A. Uluc, *Macprudential policy in an agent-based model of the UK housing market* (2016).
- [13] G. Catapano, F. Franceschi, M. Loberto, V. Michelangeli, *Macprudential policy analysis via an agent based model of the real estate sector*, *Temi di Discussione (Working Paper)* 1338, Bank of Italy (2021).
- [14] S. Poledna, E. Rovenskaya, J. Crespo Cuaresma, S. Kaniovski, M. Miess, Recovery of the Austrian economy following the COVID-19 crisis can take up to three years, *IIASA Policy Brief* (26) (2020).
- [15] J. Grazzini, C. H. Hommes, S. Poledna, Y. Zhang, Understanding post-pandemic inflation dynamics with a behavioral macroeconomic model of the Canadian economy, Available at SSRN 4381235 (2023).
- [16] C. H. Hommes, S. Poledna, Analyzing and forecasting economic crises with an agent-based model of the euro area, Available at SSRN 4381261 (2023).
- [17] S. Poledna, N. Strelkovskii, A. Conte, A. Goujon, J. Linnerooth-Bayer, M. Catalano, E. Rovenskaya, Economic and labour market impacts of migration in austria: an agent-based modelling approach, *Comparative Migration Studies* 12 (1) (2024) 18.
- [18] G. Bachner, N. Knittel, S. Poledna, S. Hochrainer-Stigler, K. Reiter, Revealing indirect risks in complex socioeconomic systems: A highly detailed multi-model analysis of flood events in Austria, *Risk Analysis* 44 (1) (2024) 229–243.
- [19] S. Hochrainer-Stigler, G. Bachner, N. Knittel, S. Poledna, K. Reiter, F. Bosello, Risk management against indirect risks from disasters: A multi-model and participatory governance

- framework applied to flood risk in Austria, *International Journal of Disaster Risk Reduction* 106 (2024) 104425.
- [20] C. Kimmich, K. Weyerstraß, T. Czypionka, N. F. Fauster, M. Kinner, E. Laa, L. Mateeva, K. Plank, L. Ulrici, H. Zenz, et al., Economic impact of labor productivity losses induced by heat stress: An agent-based macroeconomic approach, *Research Square* (2024).
URL <https://doi.org/10.21203/rs.3.rs-4526622/v1>
- [21] A. Gill, M. Lalith, S. Poledna, M. Hori, K. Fujita, T. Ichimura, High-performance computing implementations of agent-based economic models for realizing 1: 1 scale simulations of large economies, *IEEE Transactions on Parallel and Distributed Systems* 32 (8) (2021) 2101–2114.
- [22] S. Wiese, J. Kaszowska-Mojša, J. Dyer, J. Moran, M. Pangallo, F. Lafond, J. Muellbauer, A. Calinescu, J. D. Farmer, Forecasting macroeconomic dynamics using a calibrated data-driven agent-based model, *arXiv preprint arXiv:2409.18760* (2024).
- [23] C. Hommes, M. He, S. Poledna, M. Siqueira, Y. Zhang, CANVAS: A Canadian behavioral agent-based model for monetary policy, *Journal of Economic Dynamics and Control* 172 (Mar. 2025).
- [24] D. Coletti, A Blueprint for the Fourth Generation of Bank of Canada Projection and Policy Analysis Models, *Staff Discussion Paper 2023-23*, Bank of Canada (2023).
- [25] M.-A. Gosselin, S. Kozićki, Making it real: Bringing research models into central bank projections, *Staff Discussion Paper 2023-29*, Bank of Canada (2023).
- [26] S. Poledna, et al., Economic forecasting with an agent-based model: Matlab code, accessed: 2024-11-27 (2023).
URL <https://github.com/iiasa/abm>
- [27] Simudyne, Economic forecasting model, accessed: 2025-01-13 (2024).
URL https://docs.simudyne.com/commercial_models/econ_forecast
- [28] S. Brusatin, T. Padoan, A. Coletta, D. Delli Gatti, A. Glielmo, Simulating the economic impact of rationality through reinforcement learning and agent-based modelling, in: *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 159–167.
- [29] M. Benedetti, G. Catapano, F. De Scavis, M. Favorito, A. Glielmo, D. Magnanimi, A. Muci, Black-it: A ready-to-use and easy-to-extend calibration kit for agent-based models, *Journal of Open Source Software* 7 (79) (2022) 4622.
- [30] A. Glielmo, M. Favorito, D. Chanda, D. Delli Gatti, Reinforcement learning for combining search methods in the calibration of economic ABMs, in: *Proceedings of the Fourth ACM International Conference on AI in Finance*, 2023, pp. 305–313.
- [31] D. Team, Dynare.jl: A Julia rewrite of Dynare - solving, simulating and estimating DSGE models, accessed: 2024-11-24 (2024).
URL <https://github.com/DynareJulia/Dynare.jl>
- [32] T. Kockerols, Macromodelling.jl: A Julia package for developing and solving dynamic stochastic general equilibrium models, *Journal of Open Source Software* 8 (89) (2023) 5598.
- [33] M. Del Negro, M. Giannoni, P. Li, E. Moszkowski, M. Smith, The FRBNY DSGE model meets Julia, *Tech. rep.*, Federal Reserve Bank of New York (2015).
- [34] M. Del Negro, M. Giannoni, A. Gupta, P. Li, E. Moszkowski, Forecasting with Julia, *Tech. rep.*, Federal Reserve Bank of New York (2017).
- [35] B. of Canada, StateSpaceEcon.jl: A Julia package for working with macroeconomic models, accessed: 2024-11-24 (2024).
URL <https://github.com/bankofcanada/StateSpaceEcon.jl>
- [36] D. Araujo, Open-sourced central bank macroeconomic models, Available at SSRN (2024).
- [37] Eurostat, European system of accounts - ESA 2010, Manuals and guidelines 822, Eurostat (2013).
URL <https://ec.europa.eu/eurostat/web/products-manuals-and-guidelines/-/ks-02-13-269>
- [38] G. Datseris, A. R. Vahdati, T. C. DuBois, Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity, *Simulation* 100 (10) (2024) 1019–1031.
- [39] R. Sells, Julia programming language benchmark using a flight simulation, in: *2020 IEEE Aerospace Conference*, IEEE, 2020, pp. 1–8.
- [40] S. Hunold, S. Steiner, Benchmarking Julia’s communication performance: Is Julia HPC ready or full HPC?, in: *2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, IEEE, 2020, pp. 20–25.
- [41] W.-C. Lin, S. McIntosh-Smith, Comparing Julia to performance portable parallel programming models for HPC, in: *2021 International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, IEEE, 2021, pp. 94–105.
- [42] A. Slepoy, A. P. Thompson, S. J. Plimpton, A constant-time kinetic Monte Carlo algorithm for simulation of large biochemical reaction networks, *The Journal of Chemical Physics* 128 (20) (2008) 205101. doi:10.1063/1.2919546.
- [43] F. D’Ambrosio, H. L. Bodlaender, G. T. Barkema, Dynamic sampling from a discrete probability distribution with a known distribution of rates, *Computational Statistics* 37 (3) (2022) 1203–1228.
- [44] L. O. Hafner, A. Meligrana, Exact and efficient sampling from dynamic discrete distributions with finite-precision weights, *arXiv preprint arXiv:2506.14062* (2025).
- [45] W. Moses, V. Churavy, Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients, in: H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 12472–12485.
URL <https://proceedings.neurips.cc/paper/2020/file/9332c513ef44b682e9347822c2e457ac-Paper.pdf>
- [46] W. S. Moses, V. Churavy, L. Paehler, J. Hückelheim, S. H. K. Narayanan, M. Schanen, J. Doerfert, Reverse-mode automatic differentiation and optimization of gpu kernels via enzyme, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’21*, Association for Computing Machinery, New York, NY, USA, 2021. doi:10.1145/3458817.3476165.
URL <https://doi.org/10.1145/3458817.3476165>
- [47] M. Innes, A. Edelman, K. Fischer, C. Rackauckas, E. Saba, V. B. Shah, W. Tebbutt, A differentiable programming system to bridge machine learning and scientific computing, *arXiv preprint arXiv:1907.07587* (2019).
- [48] JuliaPy, PythonCall & JuliaCall, <https://github.com/JuliaPy/PythonCall.jl>, accessed: 2024-12-16 (2024).